Golpe y prueba múltiples animaciones: Par02_02

```
<!DOCTYPE html>
<html lang="es">
<head>
   <title>AR Hit Test - Rana Animada</title>
   <meta charset="utf-8">
   <meta name="viewport" content="width=device-width, user-scalable=no, minimum-scale=1.0,
maximum-scale=1.0">
   <link type="text/css" rel="stylesheet" href="main.css">
   <style>
      body {
         margin: 0;
         overflow: hidden;
         touch-action: none;
      }
      #info {
         position: absolute;
         top: 10px;
         width: 100%;
         text-align: center;
         color: white;
         font-family: monospace;
         z-index: 100;
         pointer-events: none;
         text-shadow: 1px 1px 2px black;
      }
   </style>
</head>
<body>
   <div id="info">Apunta a cualquier superficie y toca para colocar a la rana</div>
   <script async src="https://unpkg.com"></script>
   <script type="importmap">
   {
      "imports": {
         "three": "./build/three.module.js",
         "three/addons/": "./jsm/"
      }
   }
   </script>
   <script type="module">
      import * as THREE from 'three';
      import { ARButton } from 'three/addons/webxr/ARButton.js';
      import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

      let camera, scene, renderer;
      let controller;
```

```
let reticle;
let rana;
let mixer;
let hitTestSource = null;
// VARIABLE GLOBAL: Controla la secuencia de animaciones de 0 a 11
let animacionIndex = 0;

const clock = new THREE.Clock();
init();
animate();

function init() {
   const container = document.createElement('div');
   document.body.appendChild(container);

   scene = new THREE.Scene();
   camera = new THREE.PerspectiveCamera(70, window.innerWidth / window.innerHeight, 0.01,
20);

   scene.add(new THREE.HemisphereLight(0xa5a5a5, 0x898989, 3));
   const light = new THREE.DirectionalLight(0xffffff, 3);
   light.position.set(1, 1, 1).normalize();
   scene.add(light);

   // ========== RETÍCULA (se adapta al hit test) ==========
   const reticleGeometry = new THREE.RingGeometry(0.05, 0.07, 32);
   const reticleMaterial = new THREE.MeshBasicMaterial({
      color: 0x00ff00,
      side: THREE.DoubleSide,
      transparent: true,
      opacity: 0.8
   });
   reticle = new THREE.Mesh(reticleGeometry, reticleMaterial);
   reticle.visible = false;
   reticle.matrixAutoUpdate = false;
   scene.add(reticle);

   // ========== CARGAR LA RANA ==========
   const loader = new GLTFLoader();
   loader.load('./models/gltf/Rana.glb', function (gltf) {
      rana = gltf.scene;
      rana.scale.set(3, 3, 3);
      // Rota el modelo 180 grados sobre su eje vertical
      rana.rotation.y = Math.PI;
      rana.visible = false;

      mixer = new THREE.AnimationMixer(rana);
      // Almacenamos las animaciones dentro de la rana para usarlas en onSelect
      rana.animations = gltf.animations;
```

```
            rana.traverse(function (child) {
                if (child.isMesh) {
                    child.castShadow = true;
                    child.receiveShadow = true;
                }
            });
            scene.add(rana);
        });

        // ========== RENDERER ==========
        renderer = new THREE.WebGLRenderer({ antialias: true, alpha: true });
        renderer.setPixelRatio(window.devicePixelRatio);
        renderer.setSize(window.innerWidth, window.innerHeight);
        renderer.xr.enabled = true;
        container.appendChild(renderer.domElement);

        // ========== BOTÓN AR ==========
        document.body.appendChild(ARButton.createButton(renderer, { requiredFeatures: ['hit-
test'] }));

        // ========== CONTROLADOR ==========
        controller = renderer.xr.getController(0);
        controller.addEventListener('select', onSelect);
        scene.add(controller);

        window.addEventListener('resize', onWindowResize);
    }

    function onWindowResize() {
        camera.aspect = window.innerWidth / window.innerHeight;
        camera.updateProjectionMatrix();
        renderer.setSize(window.innerWidth, window.innerHeight);
    }

    // ========== EVENTO SELECT: COLOCAR/MOVER LA RANA Y CAMBIAR
ANIMACIÓN ==========
    function onSelect() {
        if (reticle.visible && rana) {
            const position = new THREE.Vector3();
            const quaternion = new THREE.Quaternion();
            const scale = new THREE.Vector3();
            reticle.matrix.decompose(position, quaternion, scale);
            rana.position.copy(position);
            rana.quaternion.copy(quaternion);
            rana.visible = true;

            // LÓGICA DE ANIMACIÓN SECUENCIAL (0 a 11)
            if (mixer && rana.animations && rana.animations.length > 0) {
```

```
            // 1. Detener por completo la animación previa
            mixer.stopAllAction();

            // 2. Seguridad en caso de que el archivo tenga menos animaciones de las esperadas
            const clipActual = animacionIndex % rana.animations.length;

            // 3. Obtener, asignar y reproducir el clip correspondiente
            const action = mixer.clipAction(rana.animations[clipActual]);
            action.play();

            console.log("Toque registrado. Ejecutando animación índice: " + clipActual);

            // 4. Incrementar el contador cíclicamente de 0 a 11 (Vuelve a 0 en la vez 13 / índice 12)
            animacionIndex = (animacionIndex + 1) % 12;
        }
    }
}

// ========== LOOP DE ANIMACIÓN CON HIT TEST ==========
function animate() {
    renderer.setAnimationLoop(function (time, frame) {
        const delta = clock.getDelta();
        if (mixer) mixer.update(delta);

        // ========== HIT TEST ==========
        if (frame) {
            const referenceSpace = renderer.xr.getReferenceSpace();
            const session = renderer.xr.getSession();

            if (hitTestSource === null && session !== null) {
                session.requestReferenceSpace('viewer').then(function (viewerSpace) {
                    session.requestHitTestSource({ space: viewerSpace }).then(function (source) {
                        hitTestSource = source;
                    });
                });
            }

            if (hitTestSource) {
                const hitTestResults = frame.getHitTestResults(hitTestSource);
                if (hitTestResults.length > 0) {
                    const hit = hitTestResults[0];
                    const hitPose = hit.getPose(referenceSpace);
                    if (hitPose) {
                        reticle.visible = true;
                        reticle.matrix.fromArray(hitPose.transform.matrix);
                    }
                } else {
                    reticle.visible = false;
                }
```

```
                }
            }

            renderer.render(scene, camera);
        });
    }
  </script>
</body>
</html>
```

https://xrus1.github.io/Par02_02/